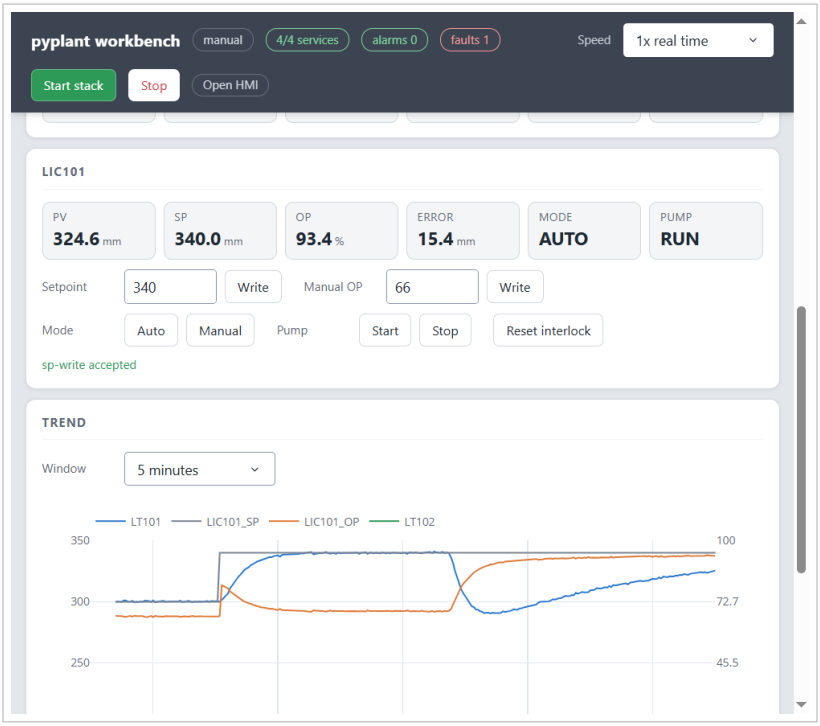
pyplant workbench

Quick user guide

in screenshots, from step 0 to a trained detector



The scenario in this guide

Unit 1 (two tanks, pump PMP101, valve FCV101, level loop LIC101) runs in automatic. We step the level setpoint from 300 to 340 mm, open a **leak in tank 1**, watch the controller hide it, confirm it on the operator HMI, and then generate data and train four detectors to catch it.

Open & run default scenario p. 2–3 → Start stack, read plant p. 4–7 → Inject leak, observe p. 8–12 → Generate dataset p. 13–15 → Train & compare p. 16–18 → Stop p. 19

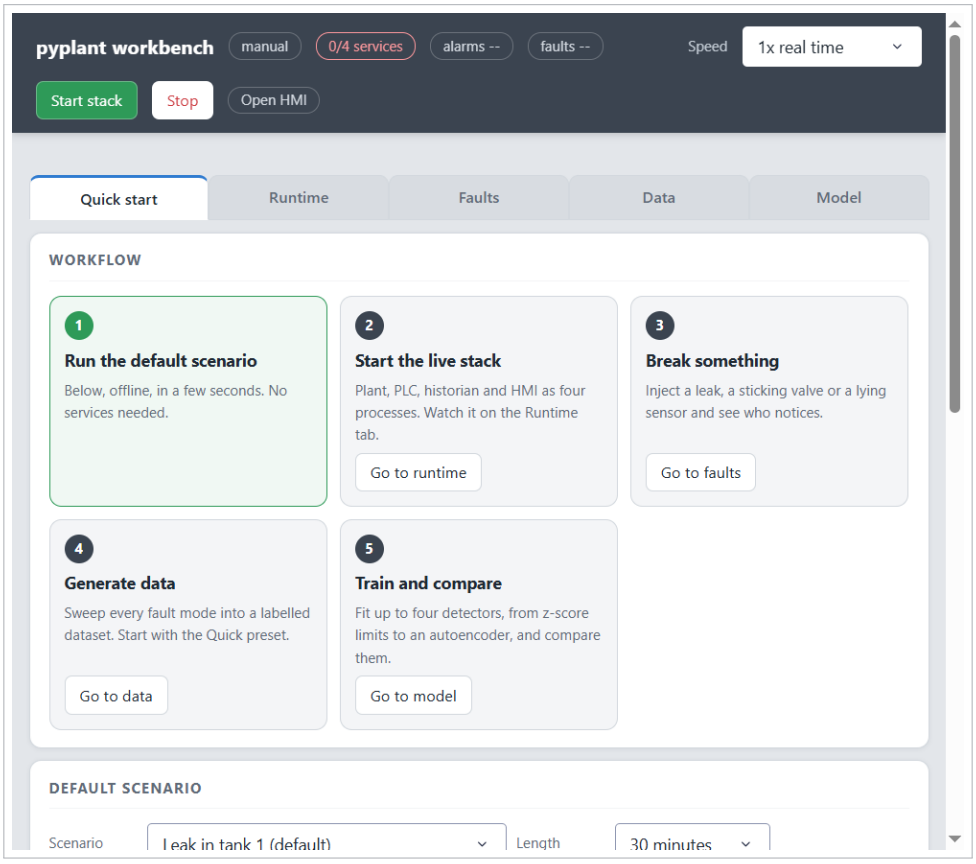
Start the workbench:

```
conda activate pyplant
python -m pyplant.workbench.app -workspace ./work/manual
```

then open `http://127.0.0.1:8050` in a browser.

Step 0 – The workbench at a glance

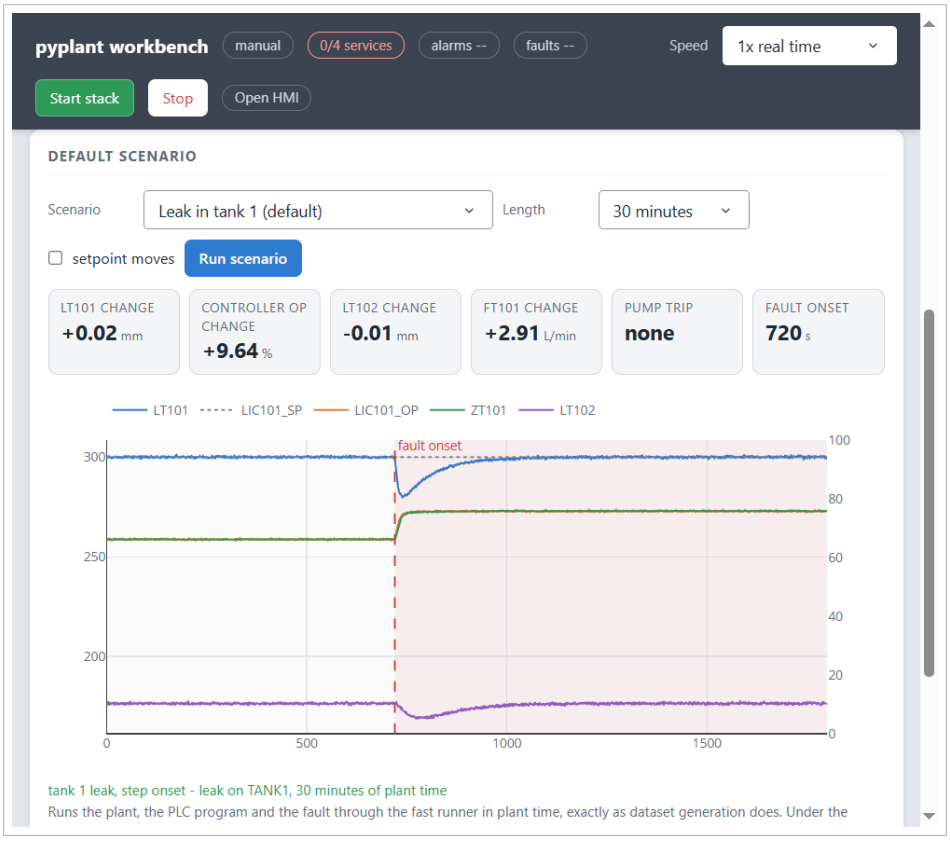
Do: Open `http://127.0.0.1:8050`. Nothing is running yet.



1. **manual** – workspace name; all files go to `work/manual`.
2. **0/4 services** – red until plant, PLC, historian and HMI are all up.
3. **alarms** / **faults** – live counts ("–" while stopped).
4. **Speed** – 1× to 20× real time, applied at the next start.
5. **Start stack** / **Stop** / **Open HMI** – control the live services.
6. Tabs **Quick start**, **Runtime**, **Faults**, **Data**, **Model**.
7. Workflow cards 1–5 – the recommended order; **Go to ...** jumps to a tab.

Step 1 – Run the default scenario (offline)

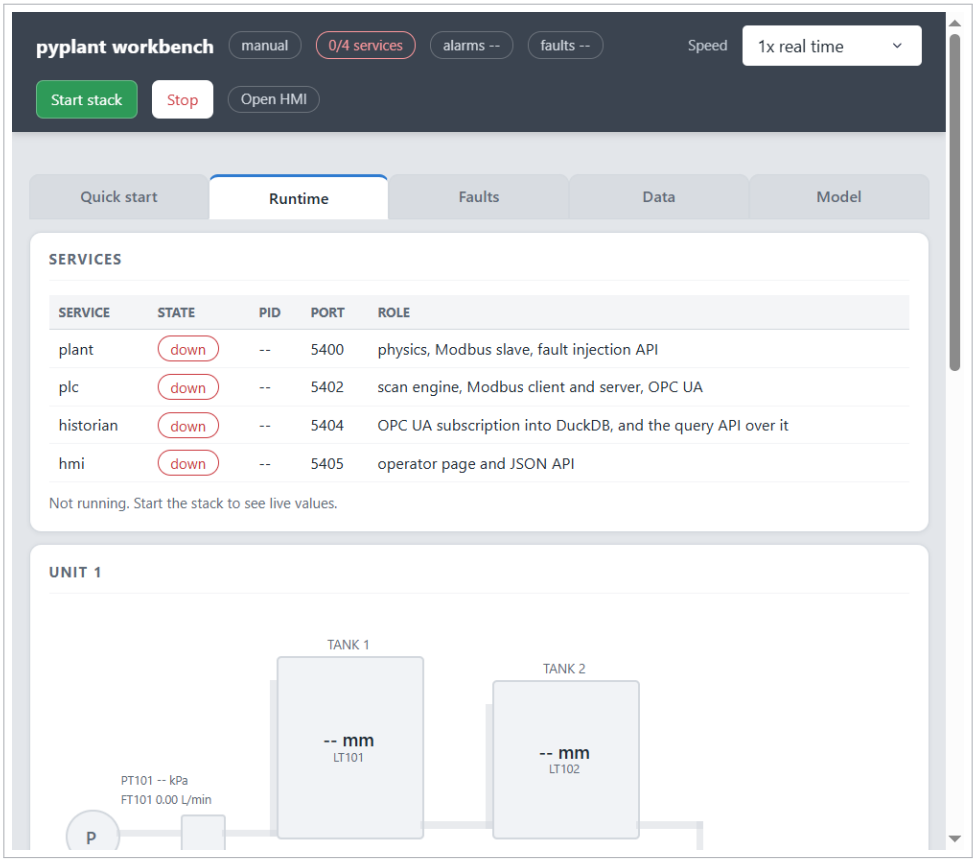
Do: Scroll to **Default scenario**, keep **Leak in tank 1**, **30 minutes**, press **Run scenario**.



1. **Scenario** and **Length** dropdowns; **setpoint moves** adds setpoint steps.
2. **Run scenario** – simulates 30 min of plant time in seconds, no services needed.
3. **LT101 CHANGE +0.02 mm** – the level hardly moves: the loop hides the leak.
4. **CONTROLLER OP CHANGE +9.64%** – the valve opens more: this is the fault signature.
5. **PUMP TRIP none**, **FAULT ONSET 720 s**.
6. Chart – levels on the left axis, OP and valve position on the right; red band = fault.

Step 2 – Runtime tab before starting

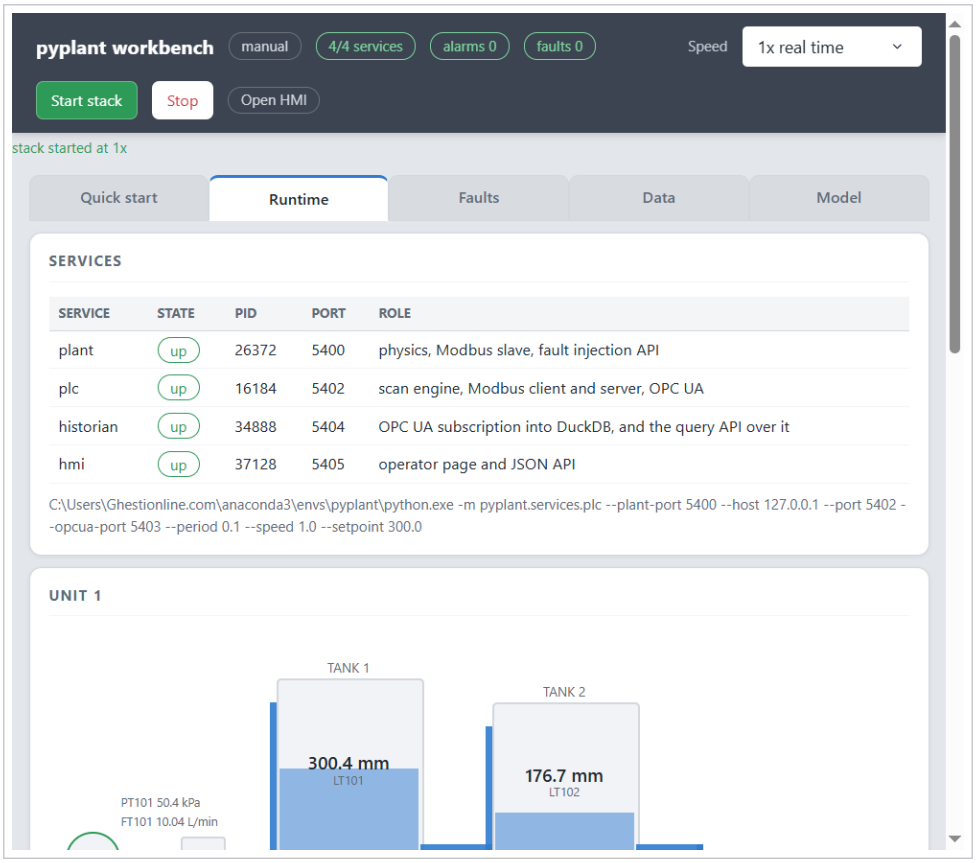
Do: Click the **Runtime** tab.



1. **SERVICES** – one row per service with **STATE**, **PID**, **PORT**, **ROLE**.
2. Red **down** pills – nothing is running.
3. **UNIT 1** mimic shows "–" until the PLC answers.

Step 3 – Start the stack

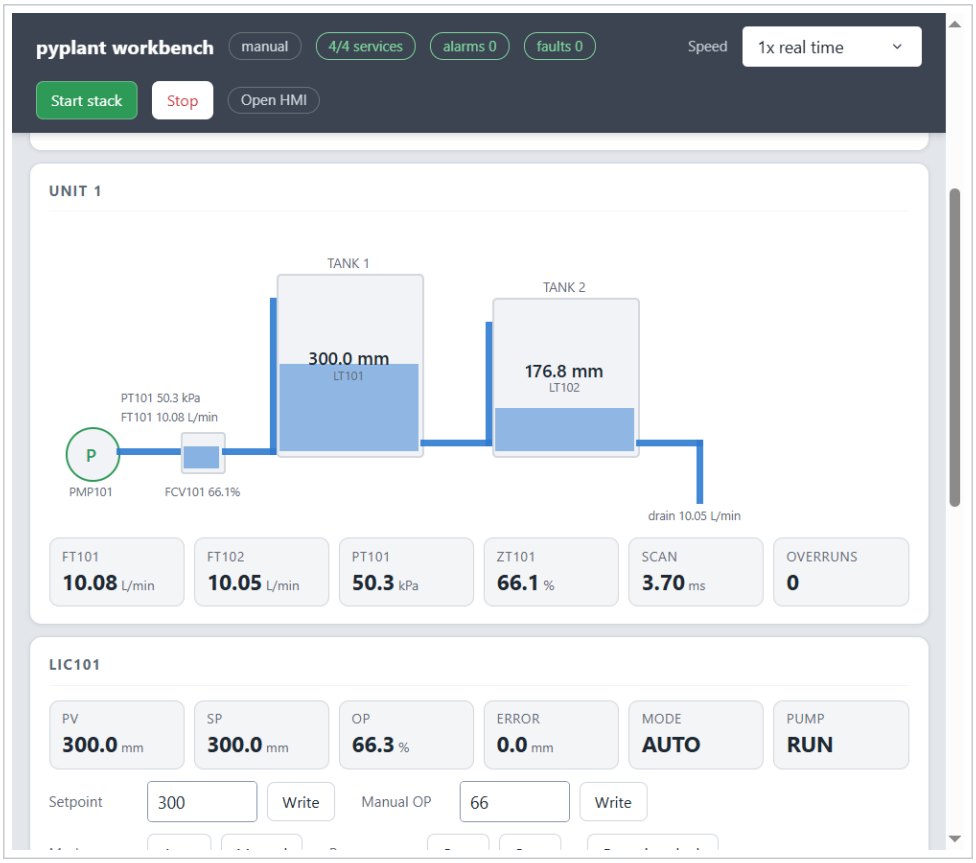
Do: Keep **Speed** at **1x real time** and press **Start stack**.



1. **stack started at 1x** – confirmation under the header.
2. **4/4 services** badge turns green; **alarms 0**, **faults 0**.
3. Every service **up**, with its PID and port (plant 5400, plc 5402, historian 5404, hmi 5405).
4. Grey line – the exact command line used, handy for running a service by hand.

Step 4 – Read the healthy plant

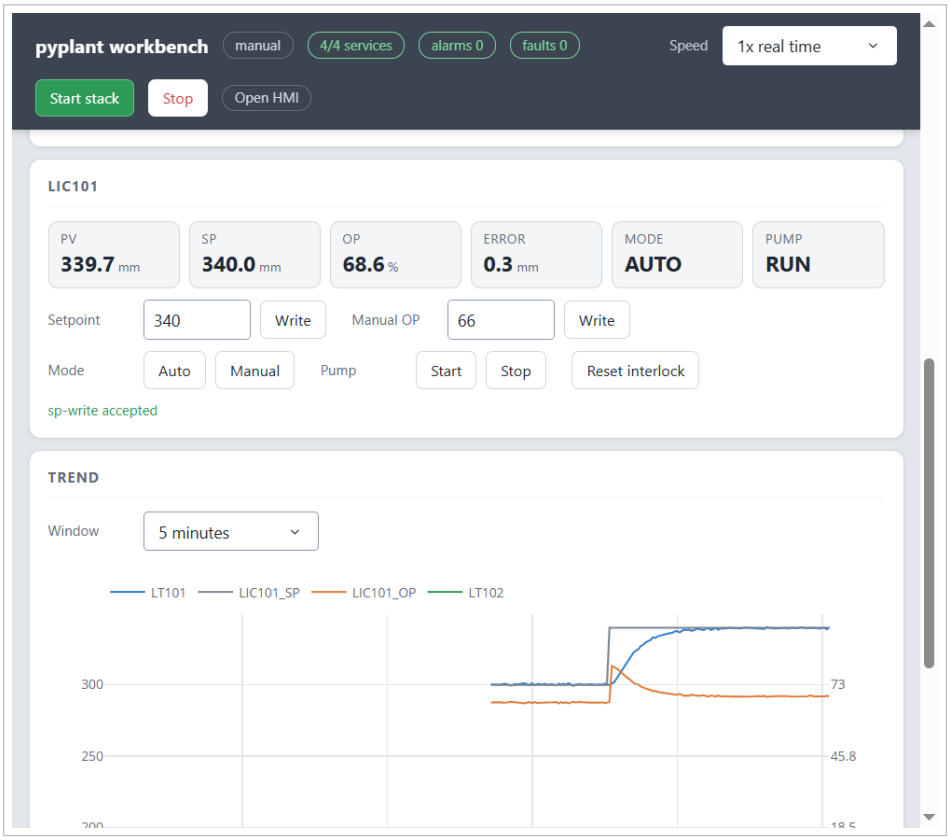
Do: Scroll to **UNIT 1**. Remember these values as the healthy reference.



1. **PMP101** pump (green = running) and **FCV101** valve at 66.1 %.
2. **TANK 1** 300.0 mm (LT101) and **TANK 2** 176.8 mm (LT102).
3. Flow tiles **FT101** 10.08 and **FT102** 10.05 L/min – in balance.
4. **PT101** pump pressure, **ZT101** valve position.
5. **SCAN** 3.70 ms of a 100 ms budget; **OVERRUNS** must stay 0.

Step 5 – Operate LIC101: setpoint step

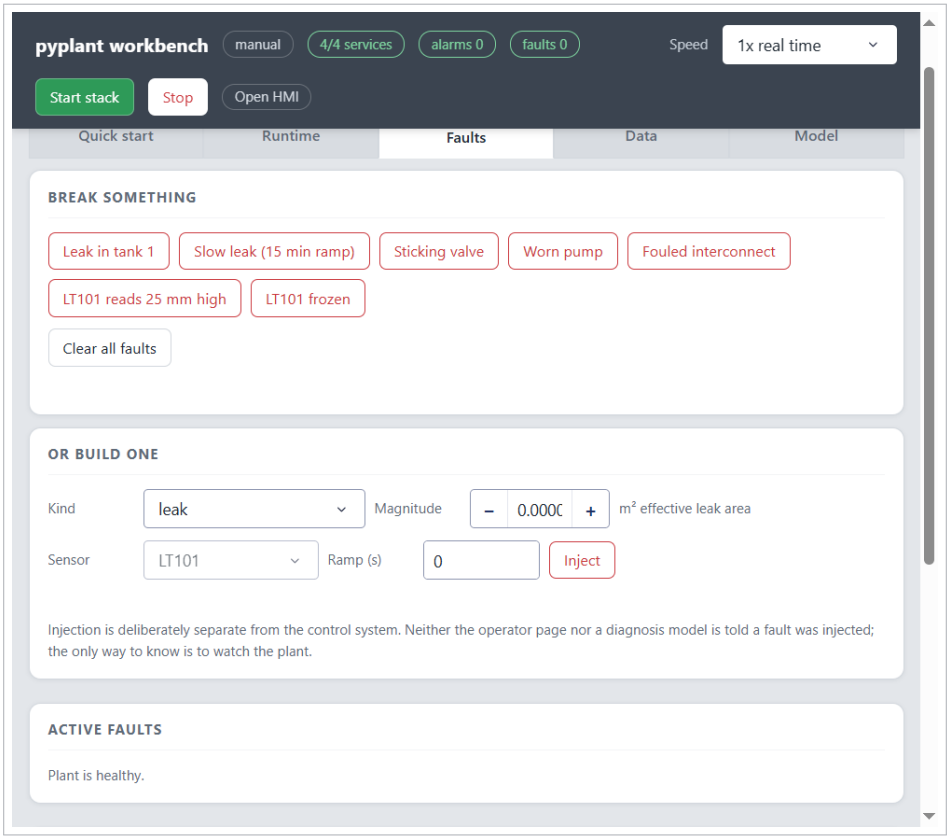
Do: Type `340` in **Setpoint**, press its **Write**, wait about 2 minutes.



1. **PV / SP / OP / ERROR** – 339.7 mm, 340.0 mm, 68.6 %, 0.3 mm.
2. **MODE AUTO**, **PUMP RUN**.
3. **Setpoint+Write**, **Manual OP+Write** (used in Manual mode only).
4. **Auto / Manual**, pump **Start / Stop**, **Reset interlock**.
5. **sp-write accepted** – the PLC took the new setpoint.
6. **TREND** – clean step: level rises to 340 mm with no overshoot.

Step 6 – The Faults tab

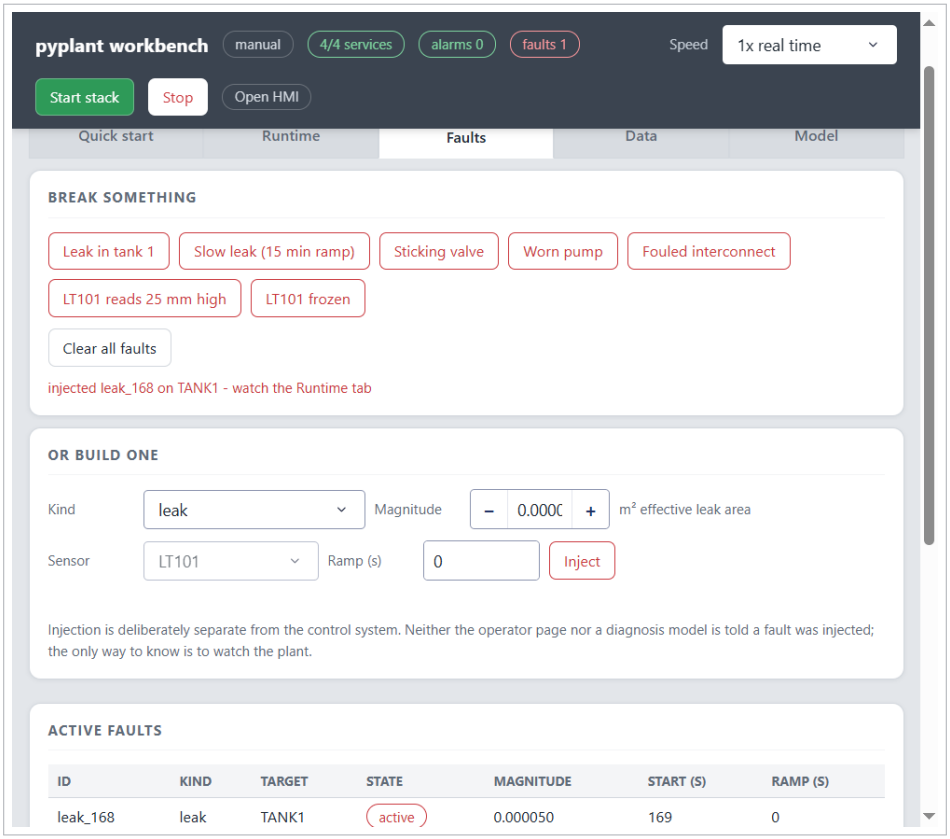
Do: Click the **Faults** tab.



1. **BREAK SOMETHING** – one-click presets (leak, slow leak, sticking valve, worn pump, fouling, sensor faults).
2. **Clear all faults** – removes every active fault.
3. **OR BUILD ONE** – **Kind**, **Magnitude** (−/+, unit shown on the right), **Sensor**, **Ramp (s)**.
4. **Inject** – applies the custom fault.
5. **ACTIVE FAULTS** – “Plant is healthy.”

Step 7 – Inject the leak

Do: Press **Leak in tank 1**.

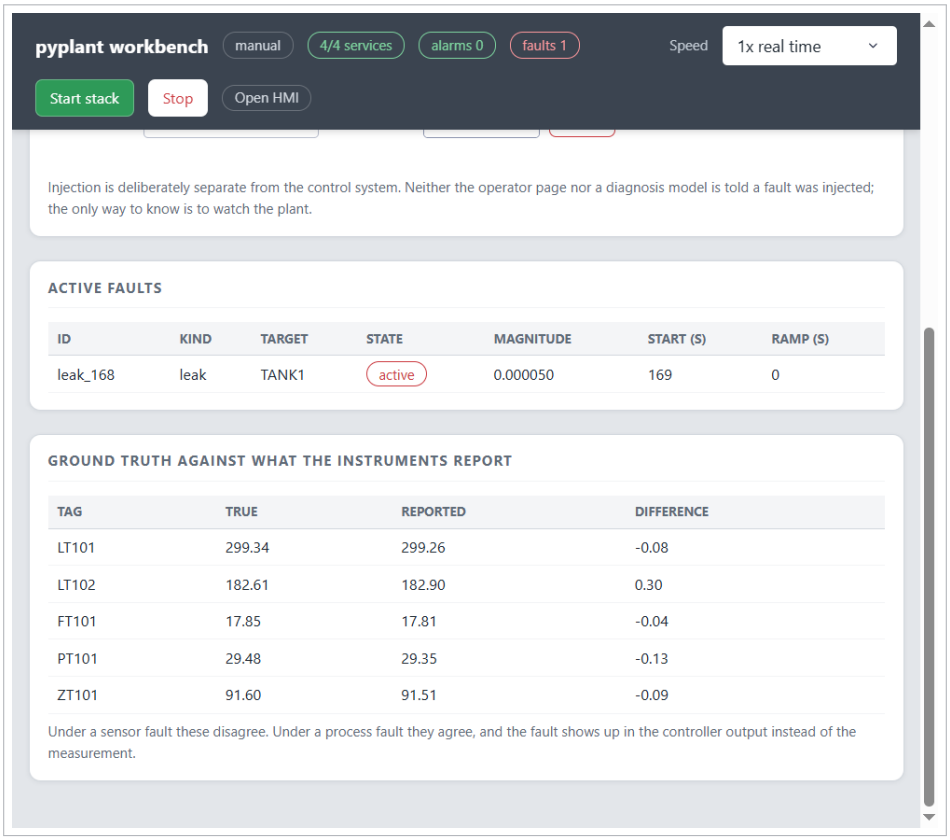


1. **faults 1** – the header badge turns red.
2. Message: **injected leak_168 on TANK1 – watch the Runtime tab**.
3. **ACTIVE FAULTS** row: leak_168, leak, TANK1, **active**, 0.000050 m^2, start 169 s, ramp 0.

Tip: Neither the HMI nor any model is told about the fault – you find it by watching the plant.

Step 8 – Ground truth vs. instruments

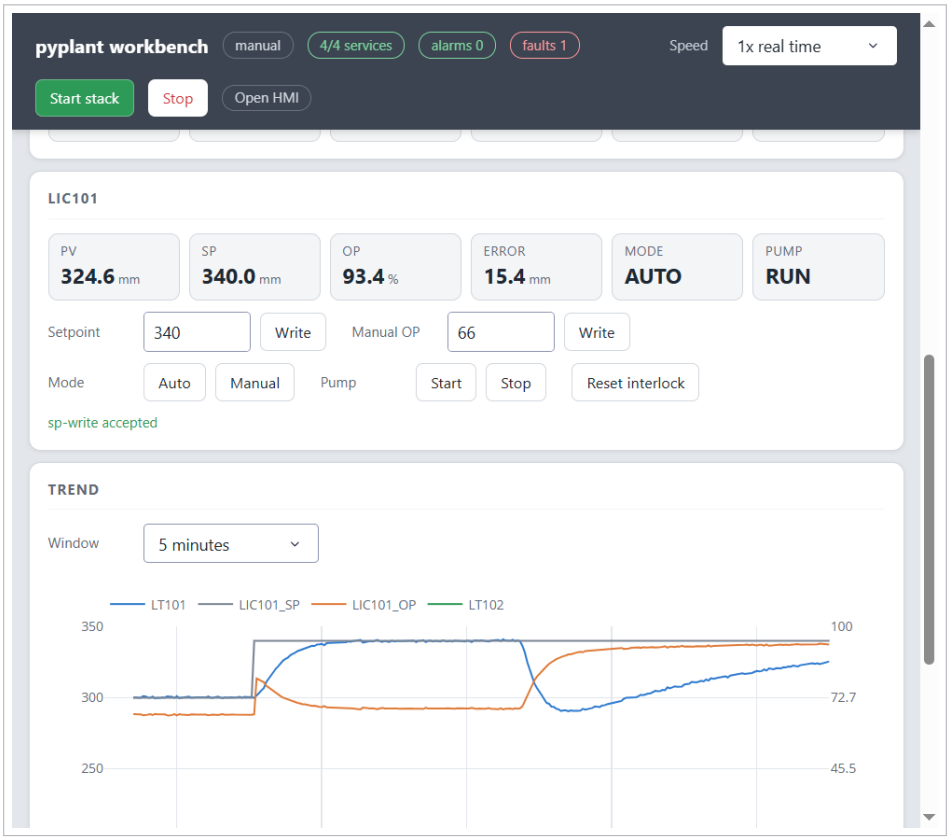
Do: Scroll down to **Ground truth against what the instruments report**.



1. **TRUE** – the real value inside the simulator.
2. **REPORTED** – what the transmitter sends.
3. **DIFFERENCE** – all within noise (<0.3): it is a *process* fault, not a sensor fault.
4. The true level, 299.3 mm, is already 40 mm under the setpoint.

Step 9 – Watch the controller fight the leak

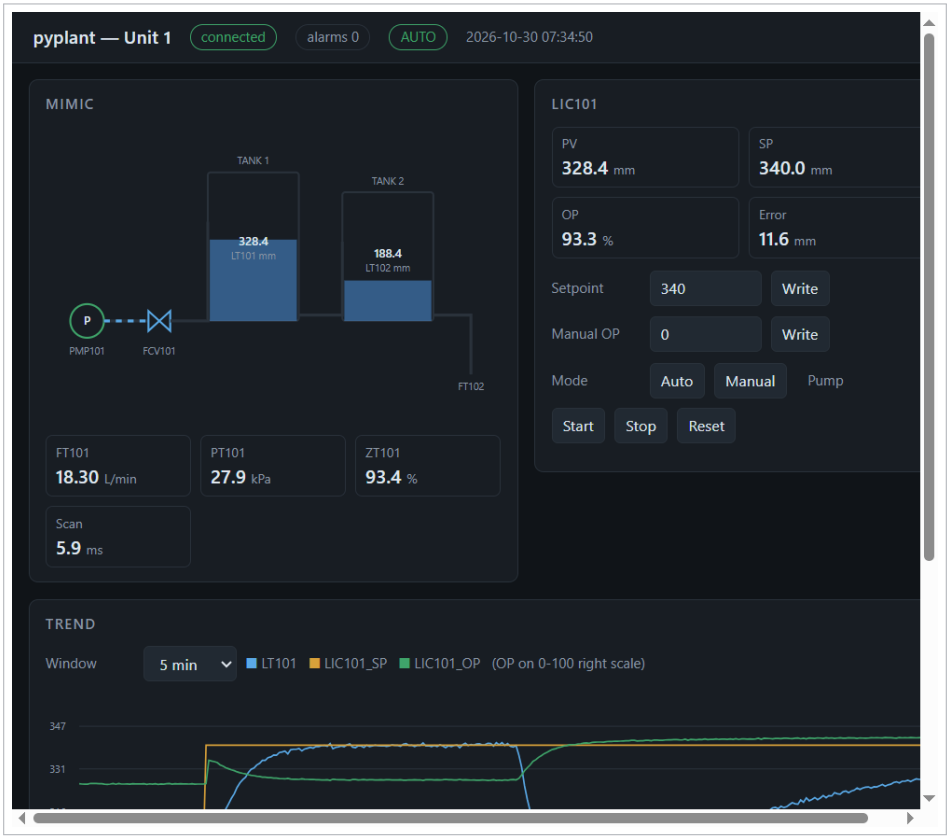
Do: Back to **Runtime**, scroll to **LIC101**, about 2 minutes after injection.



1. **PV 324.6 mm**, **ERROR 15.4 mm** – recovering, slowly.
2. **OP 93.4 %** – up from 68.6 %: the valve is nearly wide open.
3. No alarm, no trip: **AUTO**, **RUN**.
4. Trend: first the setpoint step, then the leak – level dips to ≈290 mm, OP jumps and stays high.

Step 10 – The operator HMI

Do: Click **Open HMI** (or open `http://127.0.0.1:5405`).

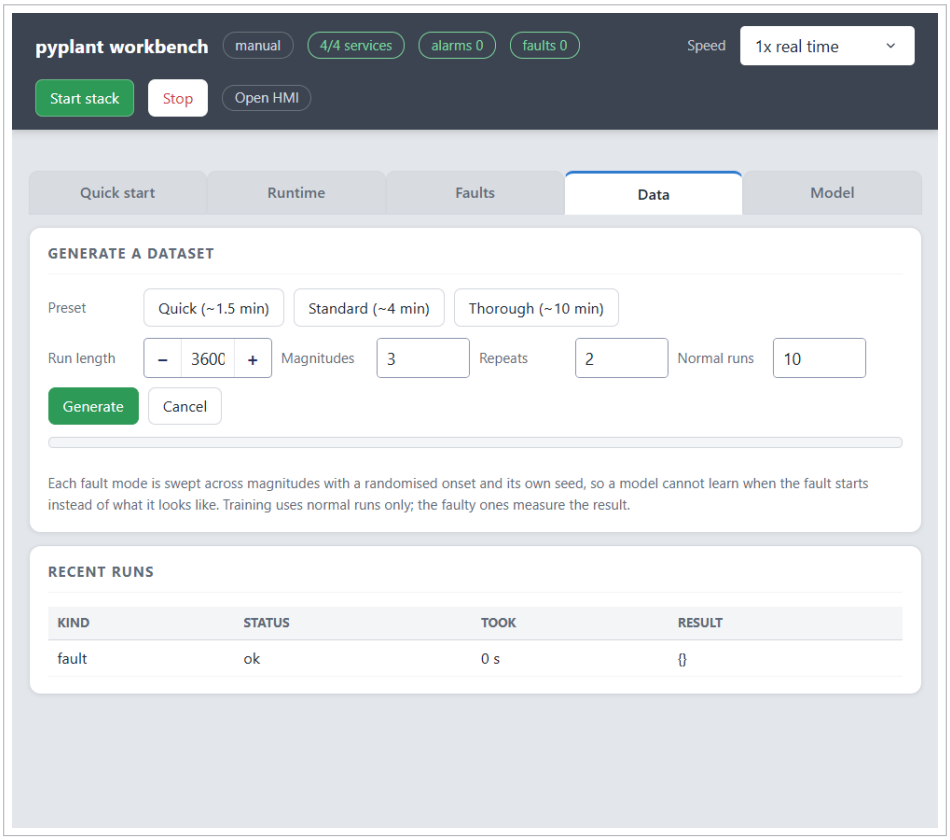


1. Header: **connected** (OPC UA live), **alarms 0**, **AUTO**, PLC time.
2. **MIMIC** in dark style; tank 1 328.4 mm, tank 2 188.4 mm.
3. **FT101 18.30 L/min** (was 10), **PT101 27.9 kPa** (was 50), **ZT101 93.4 %**.
4. **LIC101** panel – same controls as the workbench faceplate.
5. **TREND** with **Window** 2 min–1 h; **ALARMS** with **Acknowledge all** below.

Tip: The level never crossed an alarm limit. Only the extra 8 L/min of inflow and a wide-open valve give the leak away.

Step 11 – Clear the fault, open the Data tab

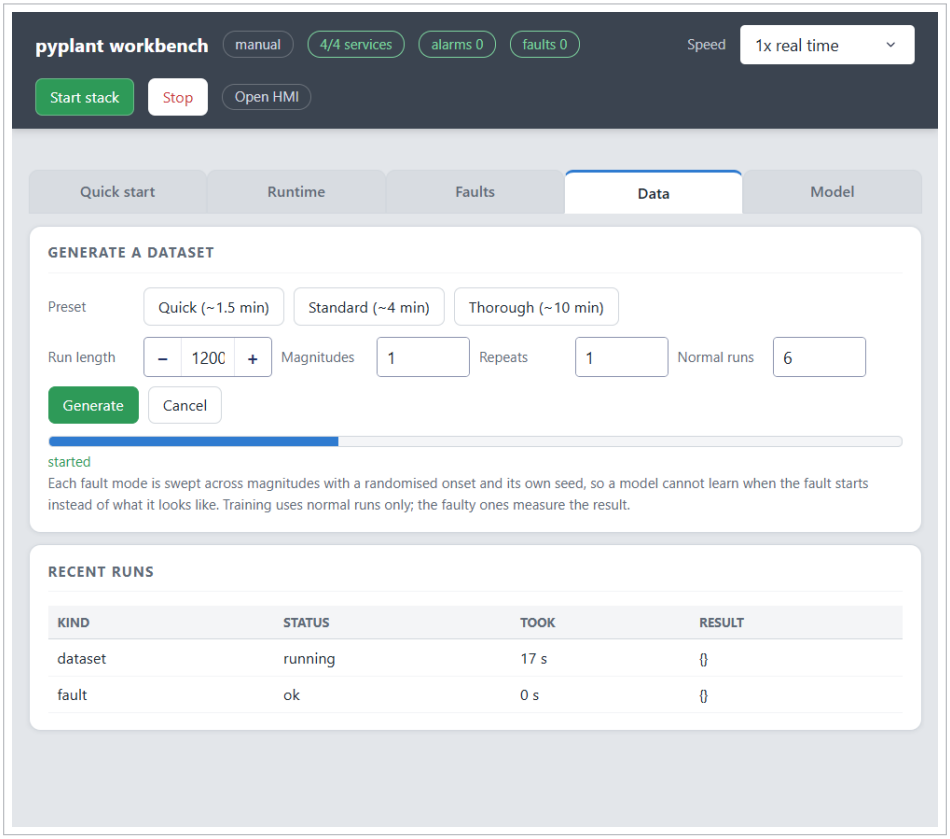
Do: On **Faults** press **Clear all faults**, then click **Data**.



1. **faults 0** – the plant is healthy again.
2. **Preset** – **Quick** (∼1.5 min), **Standard** (∼4 min), **Thorough** (∼10 min).
3. **Run length**, **Magnitudes**, **Repeats**, **Normal runs**.
4. **Generate** / **Cancel**, and the progress bar under them.
5. **RECENT RUNS** – job history (kind, status, took, result).

Step 12 – Generate a Quick dataset

Do: Press **Quick (~1.5 min)**, then **Generate**.

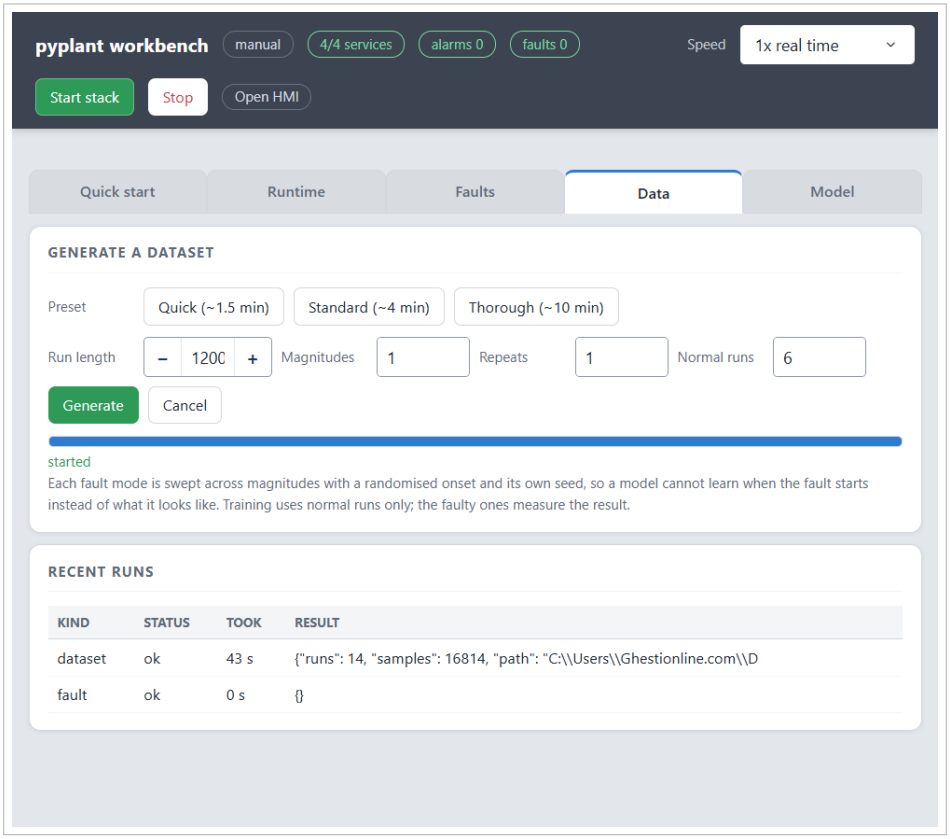


1. Fields change to 1200 s, 1 magnitude, 1 repeat, 6 normal runs.
2. **started** and the bar filling blue.
3. **dataset – running – 17 s** in **RECENT RUNS**.

Tip: Data generation uses the offline fast runner in its own process; the live plant keeps running untouched.

Step 13 – Dataset ready

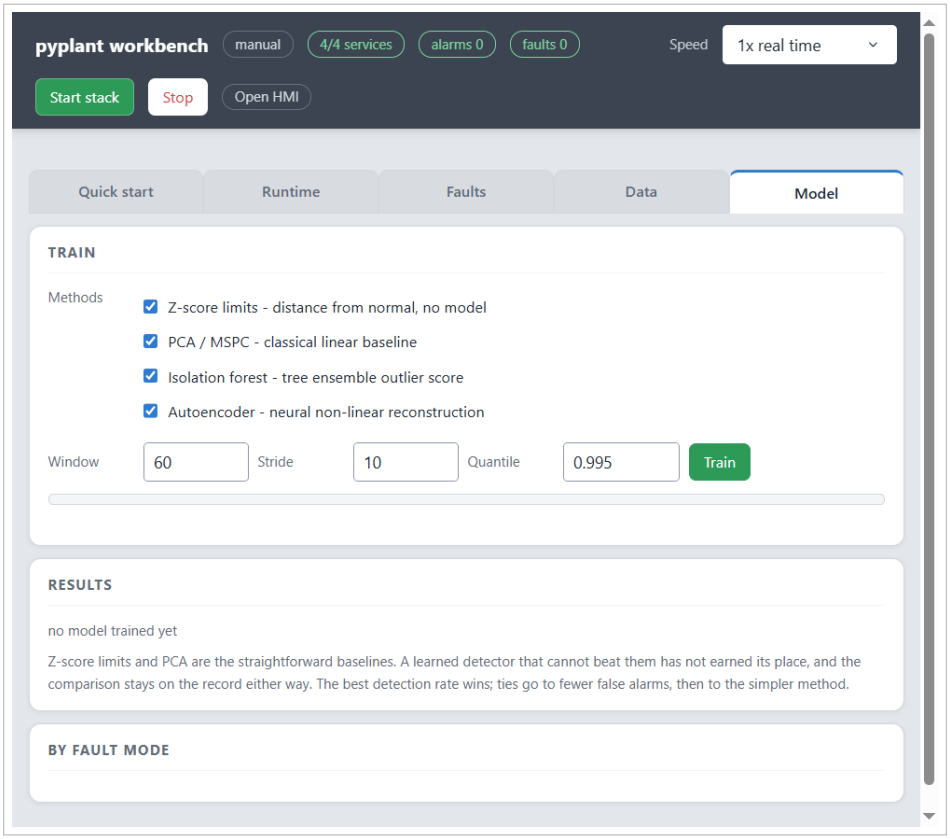
Do: Wait until the bar is full (about 45 s).



1. Full bar – job finished.
2. **dataset – ok – 43 s.**
3. Result: 14 runs (6 normal + 8 faulty), 16 814 samples, saved as `work/manual/dataset.parquet` (1.7 MB).

Step 14 – Set up training

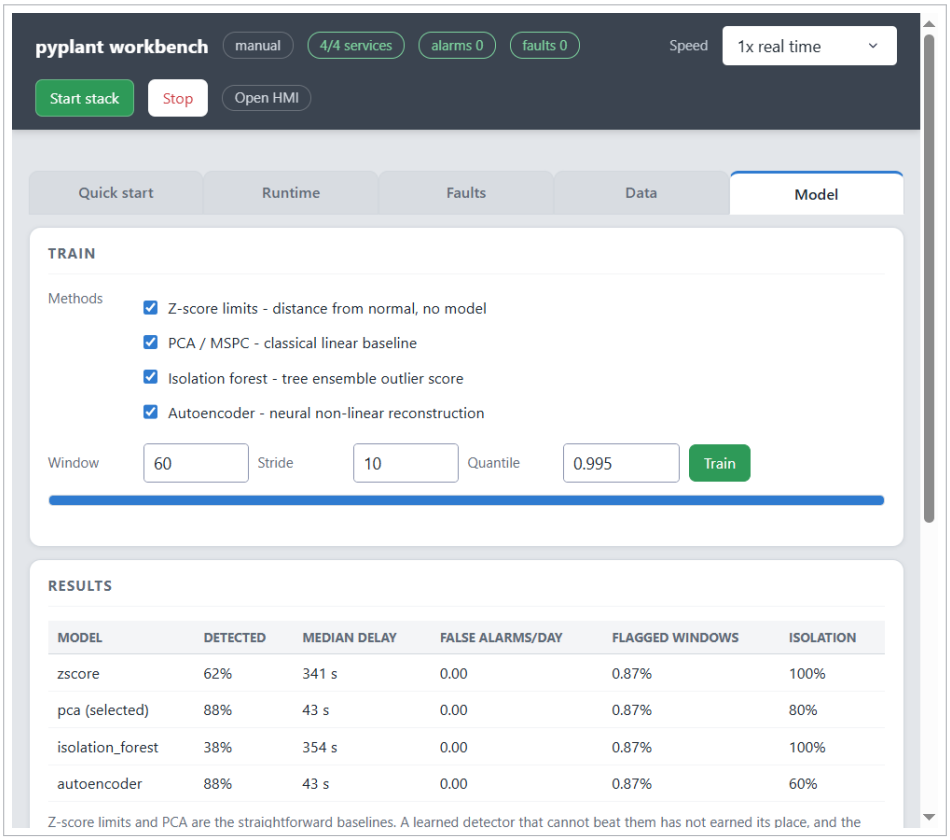
Do: Click the **Model** tab.



1. **Methods** – Z-score limits, PCA/MSPC, Isolation forest, Autoencoder (all ticked).
2. **Window** 60 samples, **Stride** 10.
3. **Quantile** 0.995 – alarm threshold on normal data; higher = fewer false alarms, later detection.
4. **Train** – starts the training job.
5. **RESULTS** – “no model trained yet”.

Step 15 – Train and compare

Do: Press **Train**. About 12 s on the Quick dataset.

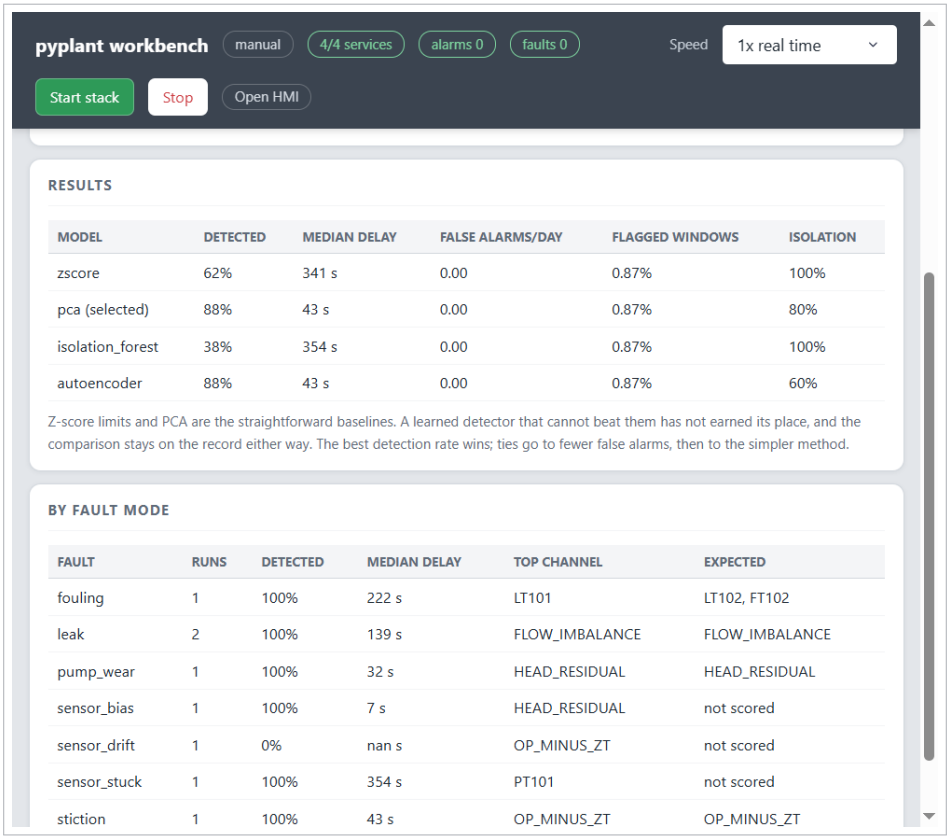


1. **detected** – z-score 62 %, **PCA 88 %**, isolation forest 38 %, autoencoder 88 %.
2. **median delay** – 43 s for PCA and autoencoder.
3. **false alarms/day** – 0.00 for all.
4. **pca (selected)** – tie with the autoencoder, so the simpler method is deployed.

Tip: If the table still says “no model trained yet” after the job ends, reload the page.

Step 16 – Results per fault mode

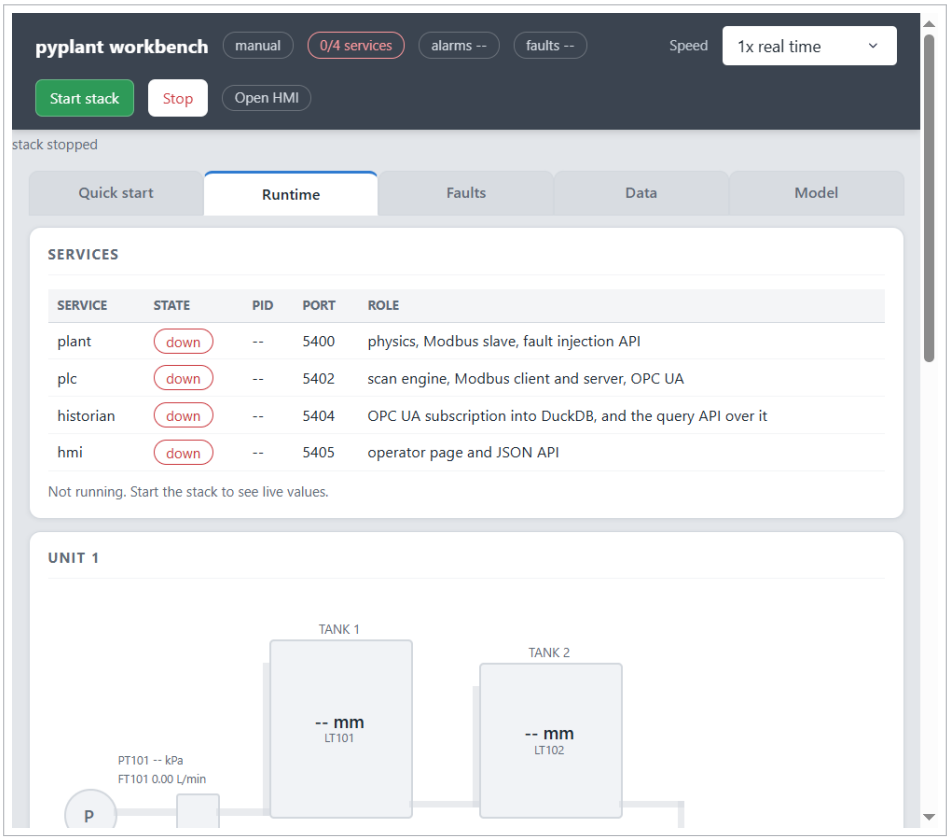
Do: Scroll to **BY FAULT MODE**.



1. One row per fault mode: **runs**, **detected**, **median delay**.
2. **leak**: 2 runs, 100 % detected, 139 s after onset.
3. **top channel** = **expected** = FLOW_IMBALANCE – the leak is also correctly isolated.
4. **not scored** – sensor faults, where isolation does not apply.

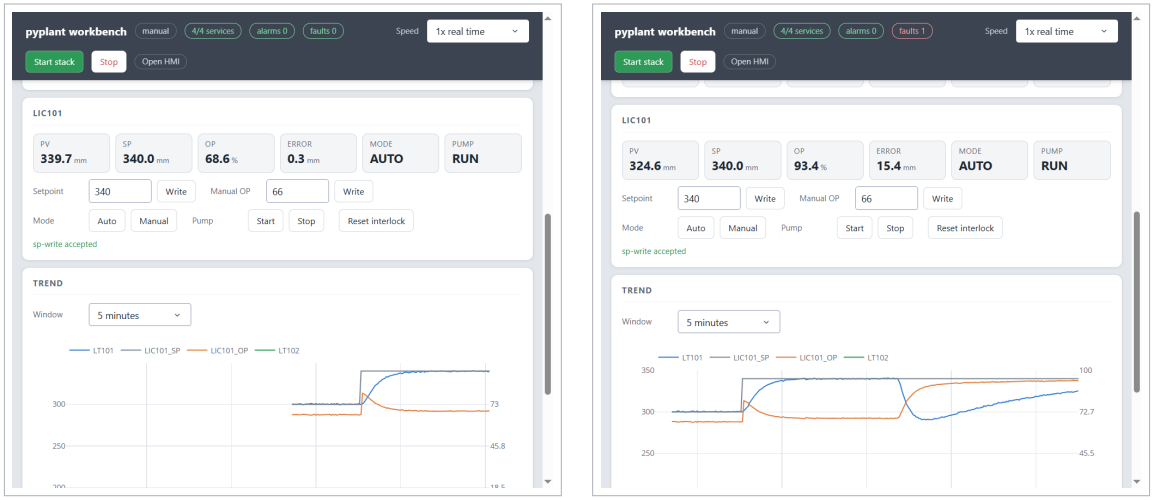
Step 17 – Stop the stack

Do: Press **Stop** in the header.



1. **stack stopped**, **0/4 services**, all rows **down**.
2. Results stay in `work/manual`: `config.json`, `history.duckdb`, `dataset.parquet`, `artifacts/detector.joblib` and `artifacts/metrics.json`.

Cheat sheet



Healthy setpoint step (left) vs. leak (right): a process fault moves **OP**, not the level.

Where things are

Workbench	`http://127.0.0.1:8050`
Operator HMI	`http://127.0.0.1:5405` (button **Open HMI**)
Ports	plant 5400, PLC 5402, OPC UA 5403, historian 5404, HMI 5405
Workspace	`-workspace ./work/<name>`; all data, models and logs inside
Service logs	`work/<name>/jobs/*.log`

Fault signatures

Leak	OP and FT101 rise, level dips then recovers, ground truth agrees
Sticking valve	ZT101 lags OP in steps, level cycles
Worn pump	PT101 and FT101 drop at the same OP
Fouled interconnect	LT101 rises relative to LT102, FT102 falls
Sensor bias / frozen	ground-truth DIFFERENCE for that tag is large

If something goes wrong

- **0/4 services** after start – a port is busy; stop old runs or change `-base-port`.
- **OVERRUNS** counting up – lower the **Speed**.
- Pump tripped – raise the level (Manual OP), then **Reset interlock** and **Start**.
- Panels stop updating – reload the page; jobs keep running in the background.
- "generate a dataset first" on **Train** – run step 12 before step 15.